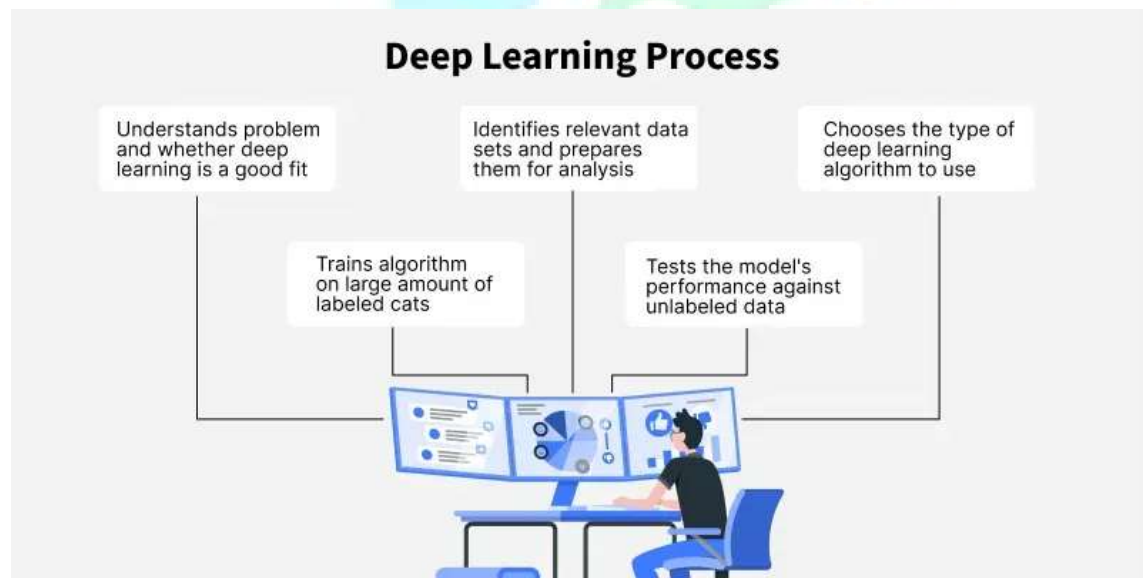


Module-05**RBT Level-L1, L2****Advancement in Computer Vision and applications:**

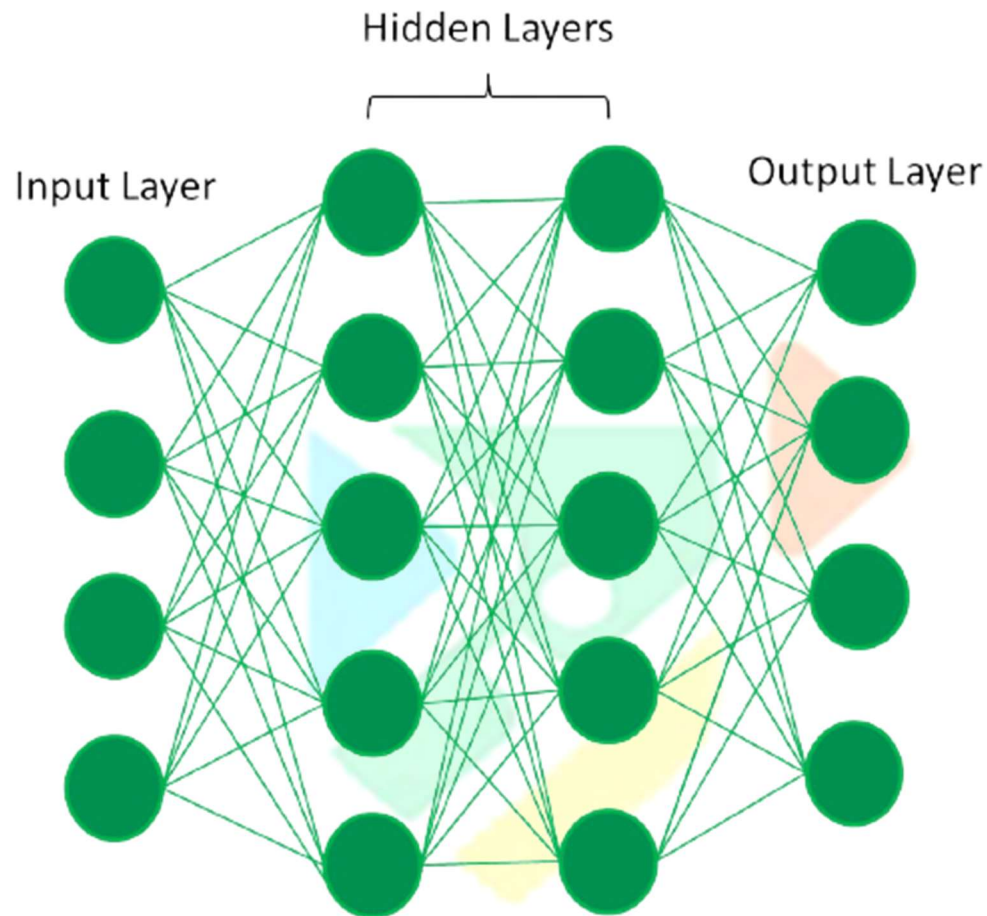
Introduction to deep learning based vision, Scene understanding and image captioning (CNN + RNN, transformers) Vision Transformers, Real time detectors, Multi object Detection, Tracking Algorithms: SORT, DeepSORT, ByteTrack (for video tracking), Object Recognition and Recent Developments, Hardware implementation.

Introduction to deep learning based vision

Deep learning is a type of machine learning that uses artificial neural networks with multiple layers to process data, mimicking how the human brain works.



In a deep neural network the input layer receives data which passes through hidden layers that transform the data using nonlinear functions. The final output layer generates the model's prediction.

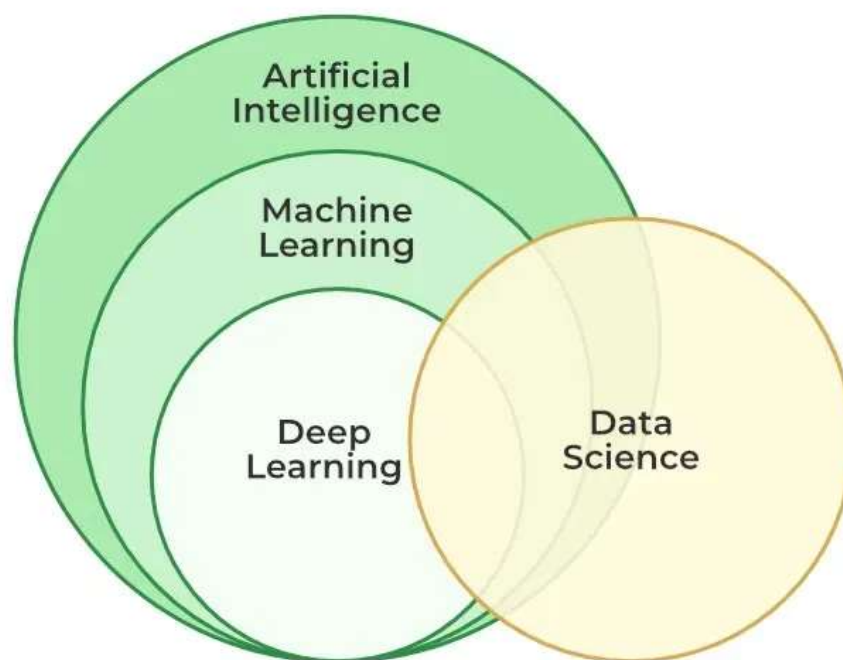


Input Layer: This is where the network receives its input data. Each input neuron in the layer corresponds to a feature in the input data.

Hidden Layers: These layers perform most of the computational heavy lifting. A neural network can have one or multiple hidden layers. Each layer consists of units (neurons) that transform the inputs into something that the output layer can use.

Output Layer: The final layer produces the output of the model. The format of these outputs varies depending on the specific task like classification, regression.

Difference between Machine Learning and Deep Learning



Machine Learning	Deep Learning
Apply statistical algorithms to learn the hidden patterns and relationships in the dataset.	Uses artificial neural network architecture to learn the hidden patterns and relationships in the dataset.
Can work on the smaller amount of dataset	Requires the larger volume of dataset compared to machine learning
Better for the low-label task.	Better for complex task like image processing, natural language processing, etc.
Takes less time to train the model.	Takes more time to train the model.
A model is created by relevant features which are manually extracted from images to detect an object in the image.	Relevant features are automatically extracted from images. It is an end-to-end learning process.
Less complex and easy to interpret the result.	More complex, it works like the black box interpretations of the result are not easy.
It can work on the CPU or requires less computing power as compared to deep learning.	It requires a high-performance computer with GPU.

Applications

1. Computer vision

- **Object detection and recognition:** Deep learning models are used to identify and locate objects within images and videos, making it possible for machines to perform tasks such as self-driving cars, surveillance and robotics.
- **Image classification:** Deep learning models can be used to classify images into categories such as animals, plants and buildings. This is used in applications such as medical imaging, quality control and image retrieval.
- **Image segmentation:** Deep learning models can be used for image segmentation into different regions, making it possible to identify specific features within images.

2. Natural language processing (NLP)

In NLP, deep learning model enable machines to understand and generate human language. Some of the main applications of deep learning in NLP include:

- **Automatic Text Generation:** Deep learning model can learn the corpus of text and new text like summaries, essays can be automatically generated using these trained models.
- **Language translation:** Deep learning models can translate text from one language to another, making it possible to communicate with people from different linguistic backgrounds.
- **Sentiment analysis:** Deep learning models can analyze the sentiment of a piece of text, making it possible to determine whether the text is positive, negative or neutral.
- **Speech recognition:** Deep learning models can recognize and transcribe spoken words, making it possible to perform tasks such as speech-to-text conversion, voice search and voice-controlled devices.

3. Reinforcement learning

In reinforcement learning, deep learning works as training agents to take action in an environment to maximize a reward. Some of the main applications of deep learning in reinforcement learning include:

- **Game playing:** Deep reinforcement learning models have been able to beat human experts at games such as Go, Chess and Atari.
- **Robotics:** Deep reinforcement learning models can be used to train robots to perform complex tasks such as grasping objects, navigation and manipulation.
- **Control systems:** Deep reinforcement learning models can be used to control complex systems such as power grids, traffic management and supply chain optimization.

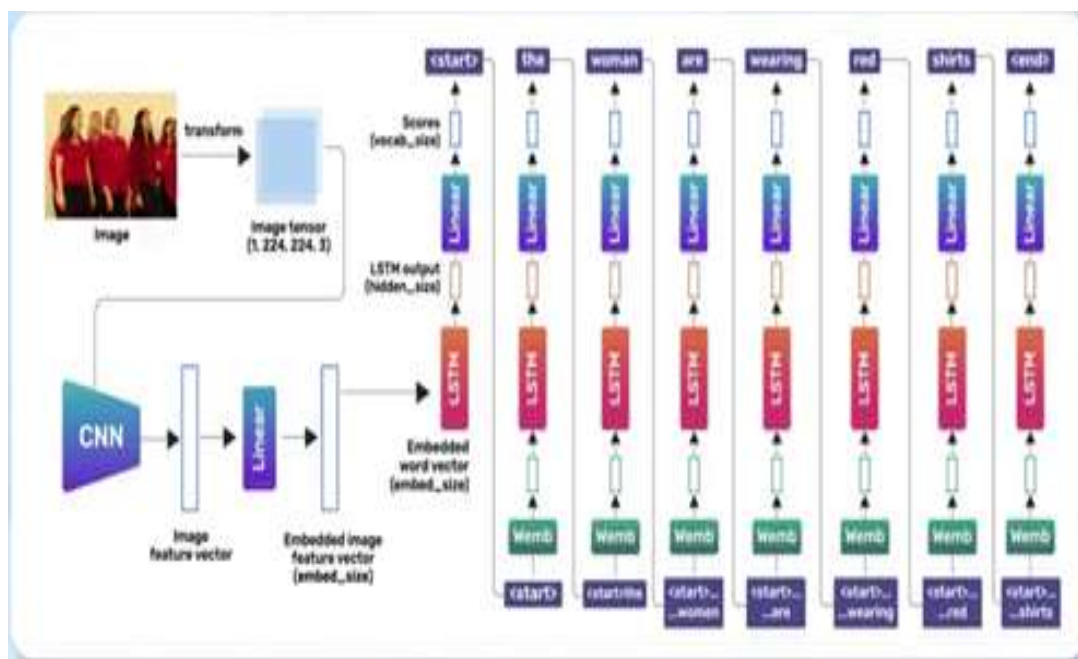
Advantages

- **High accuracy:** Deep Learning algorithms can achieve state-of-the-art performance in various tasks such as image recognition and natural language processing.
- **Automated feature engineering:** Deep Learning algorithms can automatically discover and learn relevant features from data without the need for manual feature engineering.
- **Scalability:** Deep Learning models can scale to handle large and complex datasets and can learn from massive amounts of data.
- **Flexibility:** Deep Learning models can be applied to a wide range of tasks and can handle various types of data such as images, text and speech.

Disadvantages

- **Data availability:** It requires large amounts of data to learn from. For using deep learning it's a big concern to gather as much data for training.
- **Computational Resources:** For training the deep learning model, it is computationally expensive because it requires specialized hardware like GPUs and TPUs.
- **Interpretability:** Deep learning models are complex, it works like a black box. It is very difficult to interpret the result.
- **Overfitting:** when the model is trained again and again it becomes too specialized for the training data leading to overfitting and poor performance on new data.

Scene understanding and image captioning (CNN + RNN, transformers) Vision Transformers



An image captioning system typically contains two components:

- Using a convolutional neural network (ResNet in our case) to extract visual features from the image.
- Decoding these features into a natural language sequence using a modified recurrent neural network (LSTM).

Among CNNs, ResNet (Residual Neural Network) is a strong candidate for extracting relevant features. Meanwhile, an LSTM (Long Short-Term Memory) network is a common choice for handling the language generation because it can manage long-term dependencies in language sequences.

- **Vocabulary:** Build a word vocabulary from the captions (text).
- **Dataset:** Load the Flickr8k images and captions, apply transforms, and prepare the data for training.
- **Encoder (ResNet):** Convert images into feature vectors.
- **Decoder (LSTM):** Generate captions word-by-word from the feature vectors.
- **Training:** Combine the encoder & decoder, compute loss, and optimize.
- **Inference:** A Gradio interface to run the inference with our trained model.

Real time detectors

Real-time detectors are systems that identify and locate objects in video streams or sequences of images as they are captured.

Input Image: The object detection process begins with image or video analysis.

Pre-processing: Image is pre-processed to ensure suitable format for the model being used.

Feature Extraction: The feature extractor is responsible for dissecting the image into regions and extracting features from each region.

Classification: Each image region is classified into categories based on the extracted features.

Localization: The model determines the bounding boxes for each detected object by calculating the coordinates for a box that encloses each object.

Non-max Suppression: When the model identifies several bounding boxes for the same object, non-max suppression is used to handle these overlaps.

Output: The process ends with the original image being marked with bounding boxes and labels that illustrate the detected objects and their categories.

Multi object Detection

Multi-object detection is the process of identifying and locating multiple objects of different classes within a single image or video.

Key concepts

Multi-class detection: The process of detecting multiple objects from different predefined categories in a single image.

Multi-object tracking (MOT): The task of automatically identifying multiple objects in a video and following them over time, assigning a consistent ID to each object.

Bounding box: A rectangular box drawn around a detected object that indicates its location and size.

Confidence score: A value that represents the probability of an object being detected as a certain class.

Deep Learning Methods for Object Detection

Two-Stage Detectors: These detectors work in two stages: first, they will propose candidate region and then classify the region into categories. Some of the two stage detectors are R-CNN, Fast R-CNN and Faster R-CNN.

Single-stage Detectors: In a single pass, these detectors accurately forecast the bounding boxes and class probabilities for every area of the picture.

Two-Stage Detectors for Object Detection

- **Regions with Convolutional Neural Networks:** R-CNN uses a selective search to generate around 2000 region proposals, resizes each region, extracts features with a pre-trained CNN and classifies objects within each region.
- **Fast R-CNN:** It processes the whole image with a CNN to create a feature map. ROI pooling extracts features for each region and a single network predicts both class probabilities and bounding box coordinates.
- **Faster R-CNN:** It uses a Region Proposal Network (RPN) to generate candidate object regions from feature maps. Features are pooled with ROI pooling and fed into a network that predicts object classes and bounding boxes.

Single-Stage Detectors for Object Detection

- **SSD (Single Shot MultiBox Detector):** It is a one-stage object detection model that predicts bounding boxes and class probabilities directly from feature maps of different sizes.
- **YOLO (You Only Look Once):** It is one-stage detection architecture that divides the image into a grid and predicts bounding boxes and class probabilities for each cell in one evaluation.

Applications

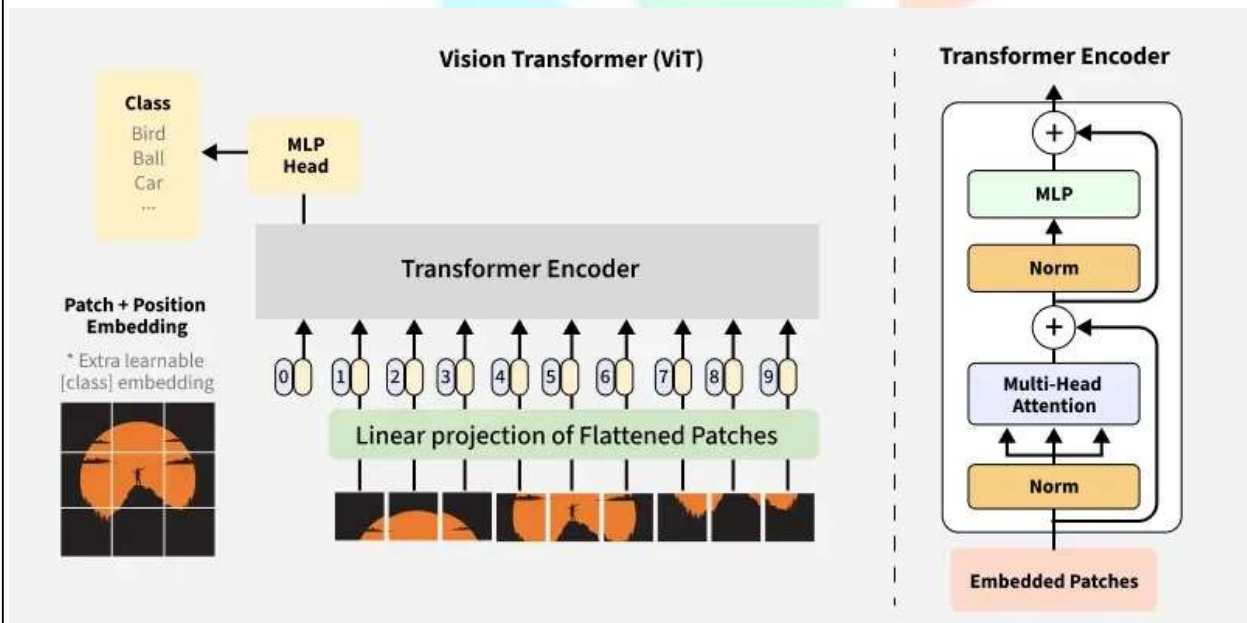
- **Autonomous Vehicles:** It can detect pedestrians other vehicles and obstacles and make real-time decisions to ensure safe navigation.
- **Security and Surveillance:** It enhances security systems by enabling the identification of suspicious activities, intruders and overall surveillance efficiency.
- **Healthcare:** It assists in medical imaging, helping to detect abnormalities such as tumors in X-rays and MRIs thus contributing to accurate and timely diagnoses.

- **Retail:** It automates inventory management, prevents theft and analyzes customer behavior enhancing operational efficiency and customer experience.
- **Robotics:** It enables robots to interact with their environment, recognize objects and perform tasks autonomously enhancing their functionality.

Vision Transformer (ViT)

Vision Transformer (ViT) is an innovative deep learning architecture designed to process visual data using the same transformer architecture that revolutionized natural language processing (NLP).

Vision Transformer (ViT) Architecture Overview



Vision Transformer architecture comprises several key stages:

- **Image Patching and Embedding**
- **Positional Encoding**
- **Transformer Encoder**
- **Classification Head (MLP Head)**

1. Image Patching and Embedding

The first and most critical step in the ViT pipeline is to convert the image into a sequence of patches, similar to the tokens in an NLP model.

- **Patch Splitting:** The input image, usually of size $H \times W \times C$ (height, width, and channels), is divided into fixed-size patches.

- **Patch Flattening:** Each patch is then flattened into a 1D vector. A patch of size $P \times P \times C$ (e.g., $16 \times 16 \times 3$) is reshaped into a vector of size $P^2 \times C$, creating 1 patch vectors for an image.
- **Patch Embedding:** Each flattened patch is projected into a higher-dimensional space (embedding dimension D) through a learnable linear projection. This linear transformation enables the model to learn richer feature representations for each patch. The result is a sequence of patch embeddings, each representing a part of the image. The total number of patches in the sequence is $N = H/P \times W/P$, where N is the number of patches.

2. Positional Encoding

Transformers do not inherently capture the spatial order of input sequences. Since the patches are processed as independent tokens, it's essential to introduce positional encodings to retain the spatial structure of the original image.

- **Positional Embedding:** Positional encodings are added to each patch embedding to encode information about the location of patches within the image. These embeddings help the model understand the spatial relationships between patches, similar to how transformers in NLP encode the positions of words in a sentence.
- **Learned vs. Fixed Positional Encoding:** In ViTs, positional encodings can either be learned during training or predefined (fixed). Most implementations of Vision Transformers use learnable positional encodings.

3. Transformer Encoder Layers

Once the patches are embedded and augmented with positional information, they are passed through a stack of transformer encoder layers. These layers consist of two primary components: Multi-Head Self-Attention (MSA) and a Feed-Forward Neural Network (FFN).

1. Multi-Head Self-Attention (MSA)

- **Self-Attention:** The self-attention mechanism allows each patch to attend to every other patch in the sequence. This means that the transformer can model long-range dependencies and relationships between different parts of the image. Each patch computes a weighted sum of the values of all other patches based on its similarity to them, known as the attention score.

- **Multi-Head Attention:** The attention mechanism is computed in parallel across multiple attention heads, allowing the model to focus on different parts of the image simultaneously.

2. Feed-Forward Network (FFN)

After self-attention, the patches are passed through a feed-forward network (FFN). The FFN consists of two fully connected layers with a non-linear activation function (typically GELU) in between.

Each transformer encoder layer includes residual (skip) connections and layer normalization to stabilize training and improve convergence. These techniques ensure that the deeper layers do not lose important information from the earlier layers.

Stacking Encoder Layers

Multiple transformer encoder layers are stacked on top of each other. Each layer refines the patch embeddings, allowing the model to build more complex and abstract representations of the image.

4. Classification Token (CLS Token)

In Vision Transformers, a special classification token (CLS token) is introduced at the beginning of the input sequence. This token serves a critical role: it gathers information from all the patches throughout the transformer layers.

The CLS token learns to represent the entire image by attending to the different patches through the self-attention mechanism. At the output of the transformer layers, the CLS token is extracted and passed to a classifier for the final prediction.

5. MLP Head (Classification Head)

After the transformer encoders process the sequence of patches and the CLS token, the output corresponding to the CLS token is used for classification.

The output of the CLS token is fed into an MLP, typically consisting of one or two fully connected layers. A softmax layer is applied at the end of the MLP for classification tasks, predicting the image's label.

Vision Transformer Architecture Summary

Input Image Processing: The input image is divided into patches, which are flattened and embedded using a linear projection.

Positional Encoding: Positional encodings are added to the patch embeddings to retain spatial information.

Transformer Encoder: The patch embeddings (along with the CLS token) are passed through multiple transformer encoder layers, which include multi-head self-attention and feed-forward networks.

Classification Head: The CLS token's output is extracted and fed into an MLP for final classification.

Tracking Algorithms: SORT, DeepSORT, ByteTrack (for video tracking), Object Recognition and Recent Developments, Hardware implementation.

Simple Online and Realtime Tracking(SORT)

In computer vision, SORT is an acronym for Simple Online and Realtime Tracking, an efficient and widely used multi-object tracking (MOT) algorithm. It is designed to track multiple detected objects through a video sequence in real-time, serving as a robust baseline for many modern tracking systems.

- **Detection:** An object detector (e.g., YOLO, Faster R-CNN) first identifies objects in each frame and provides their bounding box locations.
- **State Estimation (Kalman Filter):** For each tracked object, a Kalman filter is used to predict its position in the next frame based on its past motion. This helps to account for object movement and predict where it will likely appear.
- **Data Association (Hungarian Algorithm & IOU):** The predicted bounding boxes from the Kalman filter are then compared with the newly detected bounding boxes in the current frame. The Intersection Over Union (IOU) metric is used to measure the overlap between predicted and detected boxes. The Hungarian algorithm is then applied to optimally associate each predicted track with a new detection, minimizing the overall cost of association. This assignment process aims to maintain the identity of objects across frames.
- **Track Management:**
 - **Creation:** New tracks are initiated when a detection cannot be associated with an existing track.
 - **Deletion:** Tracks are deleted if they are not associated with any detection for a certain number of consecutive frames, indicating the object has likely left the scene or is no longer detectable.

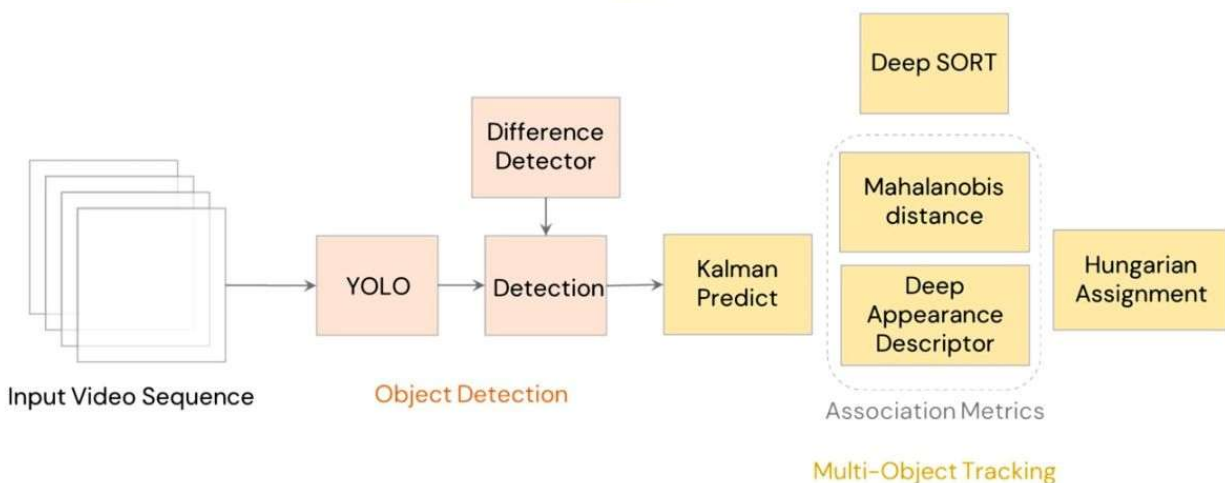
Key Features and Limitations of SORT:

- **Simplicity and Real-time Performance:** SORT's strength lies in its straightforward approach and ability to run in real-time, making it suitable for applications requiring quick processing.
- **Reliance on Motion:** SORT primarily relies on motion information (bounding box position and size) for tracking, which can lead to identity switches when objects move erratically, are occluded, or when detections are inconsistent.
- **High Identity Switches:** A significant limitation of basic SORT is its susceptibility to frequent identity switches, particularly in crowded scenes or during occlusions, as it lacks a robust mechanism to re-identify objects based on appearance.

Deep Learning-based SORT(DeepSORT)

DeepSORT (Deep Learning-based SORT) algorithm is an extension of the Simple Online and Realtime Tracking (SORT) algorithm, designed for robust multi-object tracking in video sequences. It addresses a key limitation of the original SORT algorithm by incorporating deep appearance features to reduce identity switches, particularly during occlusions or when objects appear similar.

Key Components and Functionality:



- **Object Detection:** DeepSORT typically integrates with a deep learning-based object detector (e.g., YOLO, Faster R-CNN) to identify objects and provide bounding box coordinates in each frame.
- **State Estimation (Kalman Filter):** For each tracked object, a Kalman filter is used to predict its future position and motion, helping to manage occlusions and predict where an object might reappear.

- **Appearance Feature Extraction:** This is the "Deep" part of DeepSORT. A deep convolutional neural network (e.g., MobileNet-based embedder) extracts a unique appearance descriptor (embedding) for each detected object. These embeddings capture the visual characteristics of the object, providing a "visual memory."
- **Data Association (Matching Cascade):** DeepSORT uses a cascade matching strategy to associate new detections with existing tracks. This involves:
 - **Mahalanobis Distance:** Used to measure the kinematic compatibility between predicted tracks and new detections, considering motion and position.
 - **Cosine Distance:** Used to measure the similarity between the appearance features of new detections and the appearance features stored in the track's gallery (history of appearance vectors). This is crucial for maintaining identity during occlusions.
 - **Combined Cost Function:** A weighted combination of Mahalanobis and cosine distances forms the overall cost for association, allowing for a balance between motion and appearance cues.
- **Track Management:**
 - Track Initialization:** New tracks are created for unassigned detections.
 - Track Updates:** Assigned detections update the state of existing tracks and their appearance galleries.
 - Track Deletion:** Tracks are deleted if they are not associated with any detection for a specified number of frames

Advantages of Deep SORT

- **Robustness to occlusions:** By using appearance descriptors, Deep SORT can re-identify objects after they have been occluded.
- **Discrimination of similar objects:** The appearance descriptors help distinguish between similar-looking objects.
- **Real-time performance:** Despite the added complexity, Deep SORT can operate in near real-time due to efficient feature extraction and association mechanisms.
- **Flexibility:** It can be combined with any state-of-the-art detector.

ByteTrack (for video tracking)

ByteTrack is a widely used, simple, and effective multi-object tracking (MOT) algorithm in computer vision that identifies and maintains the trajectories and identities of multiple moving objects across video frames.

ByteTrack operates as a two-stage tracking-by-detection method:

- **High-Score Association:** First, an object detection model (commonly YOLOX or YOLOv8) produces bounding boxes and confidence scores for potential objects in the current frame. Detections with a high confidence score (above a certain threshold) are associated with existing object tracks (tracklets), often using motion similarity metrics like Intersection over Union (IoU) or appearance features. A Kalman filter is used to predict the objects' next locations, aiding in this association.
- **Low-Score Association:** Detections with low confidence scores that were initially set aside are then used in a second association step with any remaining, unmatched tracklets. This step helps recover objects that might be temporarily occluded or blurred, improving tracking continuity. Unmatched low-score boxes are then typically discarded as background.
- **New Track Initialization:** Finally, any high-score detections that still remain unmatched are used to initiate new object tracks.

Key Features and Benefits

- **Improved Occlusion Handling:** By utilizing low-score detection boxes, ByteTrack is more robust in scenarios with heavy occlusion, where object detection confidence naturally decreases.
- **Simplicity and Adaptability:** The method's generic framework means it can be easily integrated with various existing object detectors (e.g., Faster R-CNN, different YOLO versions) and association metrics (IoU or Re-ID features), making it highly adaptable.
- **High Performance:** ByteTrack achieves state-of-the-art accuracy and speed, outperforming many prior methods like SORT and DeepSORT on various benchmarks, while running in real-time on single GPUs

Object Recognition and Recent Developments

Object recognition is a core computer vision task that enables machines to identify and locate objects in images or videos, essentially teaching computers to "see" and interpret the visual world like humans.

Image Classification: Assigning a single class label to an entire image (e.g., classifying an image as "dog" or "cat").

Object Localization: Locating the presence of an object in an image and marking its position with a bounding box.

Object Detection: A combination of classification and localization, where the algorithm identifies multiple objects in an image and draws a bounding box around each one. This is the most common use of the term "object recognition" in modern applications.

Instance Segmentation: A more granular approach that outlines the precise, pixel-level boundaries of each object instance, providing detailed shape and location information.

Recent Developments in Computer Vision.

- **Vision Transformers (ViTs):** ViTs have emerged as a powerful alternative to traditional CNNs, demonstrating superior performance in object detection and segmentation by processing images holistically (as a sequence of patches).
- **Real-time Performance and Efficiency:** There is a strong focus on developing models that can run in real-time on less powerful hardware (edge devices), which is crucial for applications like autonomous driving and robotics. New models like RF-DETR and the latest iterations of the YOLO (You Only Look Once) series (YOLOv9, YOLOv10, YOLOv11, and YOLOv12) are leading the charge in balancing high accuracy with minimal latency.
- **Self-Supervised Learning (SSL):** To reduce the reliance on expensive, manually labeled datasets, SSL methods have become a cornerstone of machine learning. These models learn general visual features from raw, unlabeled data, significantly cutting down on training time and cost.
- **Multi-Modal AI:** The integration of visual data with other modalities like text and audio is a major trend. Models such as OpenAI's CLIP and YOLO-World enable a richer, more context-aware understanding of scenes, allowing for applications like natural language-based object search.
- **Generative AI for Synthetic Data:** Generative Adversarial Networks (GANs) and other generative models are used to create highly realistic synthetic data. This approach helps fill data gaps for rare or sensitive scenarios (e.g., in medical imaging or autonomous vehicle edge cases) and addresses data privacy concerns.

- **Explainable AI (XAI) and Ethics:** As computer vision systems become more integrated into critical applications like healthcare and security, there is an increased emphasis on transparency and accountability. XAI techniques help ensure models are fair and that their decision-making processes can be understood by humans.

Hardware implementation.

Hardware implementation is critical for computer vision systems because it provides the necessary computational power to handle data-intensive tasks like deep learning in real time. The choice of hardware depends on factors like required performance, power consumption, cost, and physical constraints of the application.

Key Hardware Components.

- **Central Processing Units (CPUs):** General-purpose processors that manage overall system operations, data I/O, and tasks that are not suited for specialized accelerators. While essential for managing the workflow, they are less efficient for the highly parallel computations inherent in modern computer vision algorithms (e.g., convolution operations).
- **Graphics Processing Units (GPUs):** The cornerstone of modern deep learning and computer vision due to their massive parallel processing capabilities. GPUs contain thousands of cores optimized for matrix operations, which significantly accelerate both the training of large neural networks and the inference (deployment) of these models.
- **Field-Programmable Gate Arrays (FPGAs):** Reconfigurable integrated circuits that can be programmed to execute specific computer vision tasks with high efficiency and low power consumption. They offer a balance of performance and flexibility, allowing developers to customize the hardware logic after deployment for changing algorithm requirements.
- **Application-Specific Integrated Circuits (ASICs):** Custom-designed chips tailored for a specific task or application, offering the highest performance and energy efficiency for that particular workload. Examples include Google's Tensor Processing Units (TPUs) for cloud-based AI and Vision Processing Units (VPUs) like the Intel Movidius Myriad X for edge devices.
- **Vision Processing Units (VPUs):** A type of ASIC designed specifically to accelerate the processing of visual data and neural networks, often used

in low-power, embedded systems such as smart cameras, drones, and autonomous vehicles.

- **Supporting Hardware:** Beyond the main processors, other components are vital, including high-speed cameras, lenses, lighting systems, sufficient RAM, and fast storage (SSDs) to handle the large volumes of image and video data efficiently.

S.No	Questions
01	Define Deep learning & Explain layer model of deep learning.
02	Distinguish between Machine Learning and Deep Learning
03	List & explain applications of Deep learning
04	With neat block diagram Explain Scene understanding and image captioning
05	Write short note on 1) Real time detectors 2) Multi object Detection
06	List & explain the applications of Object Detection
07	Explain Deep Learning Methods for Object Detection
08	With neat block diagram Explain Architecture of Vision Transformer (ViT)
09	Explain Simple Online and Realtime Tracking(SORT)
10	With neat block diagram Explain Key Components and Functionality Deep Learning-based SORT(DeepSORT)
11	Write short note on byteTrack (for video tracking)
12	Write short note on recent Developments in Computer Vision